Agent Postmortem Generator

Part 11: Repository Architecture & Project Structure

A definitive monorepo layout and package boundary specification for a local-first AI infrastructure tool

Module purpose

Define the repository layout, package ownership rules, file-placement conventions, and dependency boundaries that keep the project small, navigable, and maintainable as it grows.

Scope summary

- Python monorepo with a single build configuration and a single primary package namespace.
- Clear separation between entrypoints, core domain code, adapters, fixtures, tests, and documentation.
- Stable conventions for root files, package names, generated artifacts, and local developer workflow.

Executive Overview

This module defines how the repository is organized, where each class of file belongs, how packages depend on each other, and what the project will not tolerate in its file layout. It is the structural contract that makes the rest of the codebase easier to reason about.

The repository is a Python monorepo with one product, one core package namespace, and a handful of clearly separated supporting areas: documentation, tests, fixtures, scripts, and developer tooling. The design intentionally avoids nested mini-repositories, duplicated tooling stacks, and ambiguous shared folders.

Its job is not to define behavior. Its job is to make behavior easy to find, easy to change, and hard to accidentally break.

Problem Statement

As the project grows, the most common failure mode is not algorithmic complexity. It is structural drift: code gets scattered, dependencies become circular, fixtures become inconsistent, and no one can tell where a change should live.

This project must remain small enough for two or three engineers to maintain without inventing a platform team. That requires a repository layout that enforces boundaries instead of merely describing them.

The repository architecture must support a deterministic pipeline, future adapters, local-only execution, and multiple output formats without turning the codebase into a pile of unrelated helpers.

Goals

- Keep the project legible from the root directory down to the module level.
- Make ownership obvious: every directory should have a single primary purpose.
- Keep the canonical trace model and downstream pipeline code isolated from UI, CLI, tests, and scripts.
- Make test fixtures, sample traces, and generated artifacts easy to find and easy to keep out of production code paths.
- Minimize cognitive overhead for contributors reviewing pull requests or tracing a bug through the system.

Non-Goals

- Designing the internal algorithms of parsing, normalization, analysis, or rendering; those belong to the module design documents for those subsystems.
- Supporting multiple languages, build systems, or package managers in the MVP.
- Creating a plugin marketplace, a multi-repo architecture, or a distributed service architecture.
- Defining product roadmap priorities, release management policy, or organizational governance beyond the repository layout itself.
- Optimizing for enterprise monorepo scale; the project should remain intentionally compact.

Repository Topology

The repository should use a single source tree with a standard Python src layout. Root-level files define project metadata and operational policy. Product code lives under one package namespace. Tests, fixtures, docs, and scripts stay outside the runtime package so they cannot be imported accidentally as application code.

```
repo-root/
├── pyproject.toml
├── README.md
├── LICENSE
├── CONTRIBUTING.md
├── CODE_OF_CONDUCT.md
├── CHANGELOG.md
├── .gitignore
├── .pre-commit-config.yaml
├── .github/
│   └── workflows/
├── docs/
│   ├── architecture/
│   ├── modules/
│   ├── decisions/
│   └── diagrams/
├── examples/
├── fixtures/
├── scripts/
├── src/
│   └── agent_postmortem/
└── tests/
```

Top-level directory responsibilities

Path	Primary purpose	Allowed contents	Do not put here
pyproject.toml	Single build and tooling source of truth	Dependency metadata, formatter/linter/test config, package metadata	Runtime code, docs prose, test fixtures
src/agent_postmortem/	Application and library code	Core domain, pipeline, adapters, CLI, UI, output writers, shared errors	Docs, sample outputs, ad hoc scripts, tests
tests/	Verification code	Unit, integration, regression, fixture-driven tests	Production code or helper code that is required at runtime
fixtures/	Reusable test and demo inputs	Canonical traces, edge cases, malformed inputs, expected outputs	Business logic or generated source artifacts
docs/	Project documentation	Architecture docs, module specs, decisions, diagrams	Runtime code or sample outputs that are not documentation
examples/	Human-readable demonstrations	Small sample runs, walkthrough inputs and outputs	Large test corpora or production fixtures
scripts/	Developer automation	Maintenance scripts, conversion helpers, local validation tools	Product logic or long-lived business logic
.github/workflows/	CI and release automation	Tests, lint, packaging, docs validation workflows	Application code or repository policy prose

Python Package Organization

The project should use a single top-level Python namespace to keep imports predictable. The package layout should reflect the layered architecture and should favor explicit names over generic buckets such as common, shared, or utils.

```
src/agent_postmortem/
├── __init__.py
├── app/
│   ├── __init__.py
│   └── orchestration.py
├── domain/
│   ├── __init__.py
│   ├── canonical_trace.py
│   ├── trace_entities.py
│   └── invariants.py
├── ingestion/
│   ├── __init__.py
│   ├── parser.py
│   ├── validation.py
│   └── normalization.py
├── analysis/
│   ├── __init__.py
│   └── engine.py
├── generation/
│   ├── __init__.py
│   └── postmortem.py
├── output/
│   ├── __init__.py
│   ├── markdown.py
│   ├── mermaid.py
│   ├── json.py
│   └── export.py
├── adapters/
│   ├── __init__.py
│   └── generic_json.py
├── cli/
│   ├── __init__.py
│   └── entrypoint.py
├── ui/
│   ├── __init__.py
│   └── app.py
├── config/
│   ├── __init__.py
│   └── settings.py
└── errors/
    ├── __init__.py
    └── exceptions.py
```

Package ownership rules

- domain/ owns the canonical trace model and shared invariants only.
- ingestion/ owns parsing, validation, and normalization helpers that turn raw inputs into canonical data.
- analysis/ owns failure-chain reasoning and first-fault identification.
- generation/ owns report assembly from analysis results and canonical trace data.
- output/ owns serialization and presentation formats, not business logic.

- adapters/ owns framework-specific or input-specific boundary code and nothing downstream.
- cli/ and ui/ are entrypoints and orchestration surfaces only; they must not contain core algorithms.
- errors/ owns exception types and structured diagnostics used across layers.

Import and dependency rules

From	May import	Must not import
domain/	Only standard library and its own submodules	CLI, UI, output writers, adapter-specific code, scripts, tests
ingestion/	domain/ and standard library	analysis/, generation/, output/, CLI/UI entrypoints
analysis/	domain/ and ingestion/ types	CLI/UI, docs, scripts, test-only helpers
generation/	domain/ and analysis/	CLI/UI internals, fixture loaders, scripts
output/	domain/ and generation/	analysis internals if that creates circular ownership, UI framework code
adapters/	ingestion/ and domain/	analysis shortcuts, generation internals, output rendering decisions
cli/ and ui/	app orchestration, output/, generation/	parser internals, normalization internals, adapter-specific logic
tests/	Any production package and dedicated test fixtures	Depending on private test-only internals from production code

The dependency direction should remain inward toward the canonical model and outward toward presentation. Code should not reach sideways across layers to borrow convenience methods from unrelated packages.

Repository layout decisions that should be avoided

- Do not create a catch-all utils/ package. If something is shared, it should be shared because it has a stable domain purpose, not because nobody wanted to name it.
- Do not split the codebase into multiple Python packages unless the project grows into a real distribution problem.
- Do not mix generated output, scratch files, and source-controlled fixtures in the same directory.
- Do not let tests import from private runtime paths outside the supported package namespace.
- Do not place product code under docs/, examples/, scripts/, or tests/ just because it is convenient in the moment.

Documentation and Artifact Strategy

Documentation must be first-class and version-controlled. Architecture and module design documents belong under docs/architecture/ and docs/modules/. Decisions that affect structure or boundaries should be captured as short architecture decision records under docs/decisions/.

Generated PDFs, if retained, should be treated as release artifacts or build outputs, not as authoritative source documents. The editable source of truth should remain Markdown or the repository-native document source.

Examples should be small and curated. Fixtures should be exhaustive enough to support regression tests, but they are not a dumping ground for every trace captured during development.

Directory	Typical contents	Retention policy
docs/architecture/	System, repository, and module architecture documents	Version-controlled
docs/modules/	Per-module design specs	Version-controlled
docs/decisions/	Short ADRs explaining key structural choices	Version-controlled
examples/	Small walkthroughs and demonstration outputs	Version-controlled if stable, otherwise generated in releases
fixtures/	Deterministic test inputs and expected outputs	Version-controlled

Testing and CI Placement

The repository should keep tests separated by intent rather than by implementation detail. Unit tests, integration tests, and regression fixtures can share naming conventions, but they should remain easy to filter and run independently.

Continuous integration definitions belong in .github/workflows/. Local helper scripts that mirror CI checks belong in scripts/ so contributors can reproduce failures without reverse-engineering pipeline steps.

Area	Recommended location	Notes
Unit tests	tests/unit/	Focused on small modules and deterministic behavior
Integration tests	tests/integration/	End-to-end or multi-module path validation
Regression fixtures	tests/fixtures/ or fixtures/	Input and expected-output bundles
CI definitions	.github/workflows/	Lint, test, package, doc checks
Local helper scripts	scripts/	Should call public interfaces or repeat CI checks

Packaging and Distribution

- Use pyproject.toml as the single build definition and dependency manifest.
- Keep the package importable from a source checkout using the src layout.
- Avoid introducing a second packaging system or custom installer logic in the MVP.
- Keep release artifacts minimal: the project should ship as source plus standard Python package metadata, not as a labyrinth of bespoke installers.

Configuration and Environment Strategy

- Environment-specific configuration should live in a small config package, not scattered through the repository.
- Default local output paths should be documented and consistent, but generated runtime artifacts should not be committed.
- Repository-level ignore rules should exclude virtual environments, build artifacts, coverage data, and local export directories.

Risks and Failure Modes

- Folder drift: new code added to the wrong directory because ownership is vague.
- Import leakage: runtime code depending on tests, scripts, or docs because no boundary is enforced.
- Fixture entropy: overlapping sample data that diverges from canonical trace behavior.
- Generated artifact pollution: temporary output accidentally committed and treated as source.
- Package sprawl: premature subpackages and helper layers that add names without adding clarity.

Implementation Checklist

- Create the src/agent_postmortem/ package tree and keep it as the single runtime namespace.
- Populate root metadata files: pyproject.toml, README.md, LICENSE, CONTRIBUTING.md, CODE_OF_CONDUCT.md, .gitignore, .pre-commit-config.yaml.
- Create docs/, tests/, fixtures/, examples/, scripts/, and .github/workflows/ with clear ownership boundaries.
- Write the canonical import policy into contributor-facing documentation and enforce it through linting and review.
- Move all sample traces and expected outputs into version-controlled fixtures with stable naming.
- Keep generated runtime exports outside the repository source tree and excluded from version control.
- Ensure every new directory has a named owner and a documented purpose before it is created.

Future Expansion

- Additional adapters can be added under src/agent_postmortem/adapters/ without changing the top-level repository shape.
- If the project later produces multiple deliverables or packaging targets, distribution logic can expand under the same pyproject.toml-controlled build system.
- If the docs program grows, docs/ can split into more refined subtrees, but only after the current structure begins to strain real use.
- If benchmark or corpus management becomes significant, a dedicated benchmarks/ or corpus/ area can be introduced with explicit ownership and tooling.

Final Recommendation

This repository architecture is MVP-ready if it is kept disciplined. The most important requirement is not cleverness; it is consistency. A small, explicit, src-based Python monorepo with stable documentation, fixtures, tests, and scripts is the right foundation for a local-first debugging tool.

The only simplification still recommended is restraint: do not add new top-level directories unless they solve a concrete ownership problem. The structure should remain boring, because boring repositories are the ones people can still understand six months later.